

MODULE 2: SQL Tables, Indexes, Table Types, Modelling and ER Diagrams

Course: SWE 306 | Level 300 | Instructor: Miss Precious Oben

1. SQL Tables

In a relational database, a table is the fundamental unit of storage. Tables are collections of related data organised in rows and columns. Every table belongs to a database and must have at least one column defined before any data can be stored in it.

When creating a table, each column must have a name, a data type, and optionally one or more constraints. A column is also called a field or attribute, which is a characteristic or property that describes the data held in the table.

1.1 Data Types

Data types define the kind of data a column can hold in a database table, for example numerical data, text, or dates. Choosing the correct data type is important because it affects storage efficiency, query performance, and the integrity of the data stored.

The table below summarises the most commonly used data types in MySQL:

Data Type	Category	Description	Example Use
CHAR(n)	String	Fixed-length character string. Always uses n characters of storage, padding with spaces if shorter.	Country code: CHAR(2)
VARCHAR(n)	String	Variable-length character string. Uses only the space needed, up to n characters.	Name, email: VARCHAR(100)
TEXT	String	Large variable-length text with no defined size limit. Cannot be indexed directly.	Descriptions, bios
ENUM(...)	String	Stores one value from a predefined list. Enforces a fixed set of allowed values.	Status: ENUM('Active','Inactive')
TINYINT(1)	Numeric	Very small integer. Used as a boolean flag where 1 means true and 0 means false.	is_active, is_verified
INT	Numeric	Standard whole number. Used for IDs, counts, and any integer value.	student_id INT
DECIMAL(p,s)	Numeric	Exact decimal number. p is total digits, s is digits after the decimal point.	salary DECIMAL(10,2)
DATE	Date/Time	Stores a date in YYYY-MM-DD format.	date_of_birth DATE

DATETIME	Date/Time	Stores a date and time in YYYY-MM-DD HH:MM:SS format.	appointment_time DATETIME
TIMESTAMP	Date/Time	Similar to DATETIME but automatically records the current date and time when a row is created.	created_at TIMESTAMP

Note: *CHAR* is best for fixed-length values such as codes and identifiers. *VARCHAR* is best for names and emails that vary in length. *TEXT* should only be used for long content such as descriptions or articles. *DECIMAL* should always be used for money instead of *FLOAT* to avoid rounding errors.

1.2 Constraints

Constraints are rules enforced on data in a database to ensure accuracy, reliability, and integrity. They are used to enforce business rules on the data and can be specified when a table is created or when it is altered later.

Constraints can be applied at two levels:

- **Column-level constraints:** Apply to a single column and are defined directly within the column definition. They enforce rules on the values that can be stored in that specific column.
- **Table-level constraints:** Apply across the whole table and are defined separately from the column definitions, typically at the end of the CREATE TABLE statement. They are used when a rule involves multiple columns or relationships between tables.

Constraint	Level	Description	Example
PRIMARY KEY	Column or Table	Uniquely identifies each row. Cannot be NULL. Each table can have only one.	student_id INT PRIMARY KEY
NOT NULL	Column	Ensures a column cannot be left empty. A value must always be provided.	first_name VARCHAR(50) NOT NULL
UNIQUE	Column or Table	Ensures no two rows have the same value in that column.	email VARCHAR(100) UNIQUE
CHECK	Column or Table	Restricts the values allowed in a column using a condition.	CHECK(age >= 18)
DEFAULT	Column	Assigns a default value when no value is provided during INSERT.	created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
FOREIGN KEY	Table	Links a column in one table to the primary key of another. Enforces referential integrity.	FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
AUTO_INCREMENT	Column	Automatically generates a unique incrementing integer for each new row.	id INT PRIMARY KEY AUTO_INCREMENT

		Used with primary keys.	
--	--	-------------------------	--

1.3 CREATE TABLE Syntax

The general structure for creating a table with data types and constraints is as follows:

```
CREATE TABLE table_name (  
    column_name1 datatype [column-level constraint(s)],  
    column_name2 datatype [column-level constraint(s)],  
    ...  
    [table-level constraint(s)]  
);
```

The example below creates an Employees table with both column-level and table-level constraints:

```
-- Column-level constraints example  
CREATE TABLE Employees (  
    ID INT PRIMARY KEY AUTO_INCREMENT,  
    FirstName VARCHAR(100) NOT NULL,  
    LastName VARCHAR(100) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Email VARCHAR(255) UNIQUE,  
    StartDate DATE DEFAULT (CURRENT_DATE)  
);
```

The following example shows a second table with table-level constraints including a FOREIGN KEY and a multi-column CHECK:

```
-- Table-level constraints example  
CREATE TABLE EmployeeProjects (  
    EmployeeID INT,  
    ProjectID INT,  
    StartDate DATE,  
    EndDate DATE,  
    PRIMARY KEY (EmployeeID),  
    FOREIGN KEY (EmployeeID) REFERENCES Employees (ID),  
    CHECK (EndDate >= StartDate)  
);
```

Note: The ORDER for defining a column is always: column name, then data type, then constraint(s). Constraints come last and multiple constraints can be stacked on a single column.

2. Querying and Filtering Data

Once data has been inserted into a table, SQL provides a rich set of tools for retrieving and filtering records. The SELECT statement is the foundation of all data retrieval in SQL.

2.1 Basic SELECT

```
-- Retrieve all columns from a table  
SELECT * FROM users;
```

```
-- Retrieve specific columns only
SELECT first_name, last_name, balance FROM users;

-- Use an alias to rename a column in the output
SELECT first_name AS 'First Name', balance AS 'Account Balance' FROM
users;
```

2.2 DISTINCT

DISTINCT is used to return only unique values in a column, removing duplicate rows from the result. It is very useful when exploring a dataset to understand what categories of data exist.

```
-- See all unique transaction types
SELECT DISTINCT transaction_type FROM transactions;

-- See all unique cities where users are located
SELECT DISTINCT city FROM users;

-- Count the unique values: combine DISTINCT with COUNT
SELECT COUNT(DISTINCT city) AS unique_cities FROM users;
```

2.3 WHERE and Comparison Operators

The WHERE clause filters rows based on a condition. Comparison operators are used to define those conditions.

Operator	Meaning	Example
=	Equal to	WHERE status = 'SUCCESS'
!= or <>	Not equal to	WHERE status != 'FAILED'
>	Greater than	WHERE amount > 50000
<	Less than	WHERE balance < 1000
>=	Greater than or equal to	WHERE credits >= 3
<=	Less than or equal to	WHERE age <= 25

2.4 BETWEEN

BETWEEN filters rows where a value falls within an inclusive range. It works with numbers, dates, and text.

```
-- Transactions between 5000 and 50000 FCFA (inclusive of both endpoints)
SELECT * FROM transactions WHERE amount BETWEEN 5000 AND 50000;

-- Users created between two dates
SELECT * FROM users WHERE created_at BETWEEN '2022-01-01' AND '2023-12-31';
```

2.5 LIKE and NOT LIKE

LIKE is used for pattern matching on text columns. Two wildcard characters are used with LIKE:

- **% (percent):** Matches any sequence of zero or more characters.
- **_ (underscore):** Matches exactly one character.

Pattern	Meaning	Example match
'P%'	Starts with P	Paul, Precious, Patrick
'%son'	Ends with son	Johnson, Jackson
'%an%'	Contains an anywhere	Fon, Tanyi, Emmanuel
'P____'	P followed by exactly 3 chars	Paul, Pere

```
-- Users whose first name starts with A
SELECT * FROM users WHERE first_name LIKE 'A%';

-- Users with an email from a specific domain
SELECT * FROM users WHERE email LIKE '%@gmail.com';

-- Transactions whose description mentions 'Transfer'
SELECT * FROM transactions WHERE description LIKE '%Transfer%';

-- NOT LIKE: exclude a pattern
SELECT * FROM users WHERE email NOT LIKE '%@lmu.edu';
```

2.6 IN and NOT IN

IN checks whether a value matches any value in a list. It is cleaner than writing multiple OR conditions.

```
-- Show SEND_MONEY and DEPOSIT transactions only
SELECT * FROM transactions
WHERE transaction_type IN ('SEND_MONEY', 'DEPOSIT');

-- Users in Buea or Limbe
SELECT * FROM users WHERE city IN ('Buea', 'Limbe');

-- Transactions that are NOT failed or pending
SELECT * FROM transactions
WHERE status NOT IN ('FAILED', 'PENDING');
```

2.7 AND, OR and NOT

Logical operators combine multiple conditions in a WHERE clause. AND requires all conditions to be true. OR requires at least one condition to be true. NOT reverses a condition. When mixing AND and OR, always use brackets to make the logic clear and avoid unintended results.

```
-- Active users with a balance above 100000
SELECT * FROM users WHERE is_verified = 1 AND balance > 100000;

-- Users from Buea or Douala
SELECT * FROM users WHERE city = 'Buea' OR city = 'Douala';

-- Use brackets when mixing AND with OR
SELECT * FROM users
WHERE (city = 'Buea' OR city = 'Douala') AND is_verified = 1;
```

2.8 IS NULL and IS NOT NULL

NULL represents a missing or unknown value. It is not the same as zero or an empty string. To check for NULL, you must always use IS NULL or IS NOT NULL. Using = NULL will never return results.

```
-- Users who have no email recorded
SELECT * FROM users WHERE email IS NULL;

-- Transactions that have an agent assigned
SELECT * FROM transactions WHERE agent_id IS NOT NULL;

-- WRONG: this will not work and returns no results
SELECT * FROM users WHERE email = NULL;
```

2.9 ORDER BY and LIMIT

ORDER BY sorts query results by one or more columns. ASC sorts from lowest to highest (default). DESC sorts from highest to lowest. LIMIT restricts how many rows are returned and is commonly used for top-N queries.

```
-- Sort transactions from largest to smallest amount
SELECT * FROM transactions ORDER BY amount DESC;

-- Top 5 highest-value transactions
SELECT * FROM transactions ORDER BY amount DESC LIMIT 5;

-- Sort users alphabetically by last name
SELECT * FROM users ORDER BY last_name ASC;
```

2.10 Aggregate Functions

Aggregate functions perform a calculation on a set of rows and return a single value. They are used together with GROUP BY to produce summaries per category.

Function	Description	Example
COUNT(*)	Counts the number of rows	COUNT(*) AS total_users
SUM(col)	Adds up all values in a column	SUM(amount) AS total_sent
AVG(col)	Returns the average value	AVG(balance) AS avg_balance
MAX(col)	Returns the highest value	MAX(amount) AS largest_tx
MIN(col)	Returns the lowest value	MIN(amount) AS smallest_tx

```
-- Count all users
SELECT COUNT(*) AS total_users FROM users;

-- Total value of all successful transactions
SELECT SUM(amount) AS total_amount
FROM transactions WHERE status = 'SUCCESS';

-- Count transactions grouped by type
SELECT transaction_type, COUNT(*) AS count
FROM transactions
GROUP BY transaction_type
ORDER BY count DESC;
```

```
-- HAVING: filter groups (used after GROUP BY, not WHERE)
SELECT city, COUNT(*) AS user_count
FROM users
GROUP BY city
HAVING COUNT(*) > 2;
```

Note: *WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY has been applied. They cannot be swapped.*

3. Indexes in Databases

An index is a special data structure that the database search engine uses to speed up data retrieval. Think of it like the index at the back of a textbook. Without it, the database reads every row from top to bottom to find a match, which is called a full table scan. With an index, the database jumps directly to the matching rows.

MySQL automatically creates an index on every PRIMARY KEY and UNIQUE column. For other columns that are frequently used in WHERE clauses or ORDER BY, you can create indexes manually.

3.1 Types of Indexes

- **Single-column index:** An index created on a single column. The most common type.
- **Composite index:** An index based on two or more columns combined. Useful when queries filter on multiple columns together.
- **Unique index:** Ensures all values in the indexed column are distinct. MySQL creates this automatically for UNIQUE columns.
- **Full-text index:** Used for full-text searches on TEXT or VARCHAR columns. More powerful than LIKE for large bodies of text.

3.2 Creating and Managing Indexes

```
-- Create a single-column index
CREATE INDEX idx_tx_date ON transactions(transaction_date);

-- Create a composite index on two columns
CREATE INDEX idx_city_network ON users(city, network);

-- Create a unique index
CREATE UNIQUE INDEX idx_unique_email ON users(email);

-- View all indexes on a table (three equivalent forms)
SHOW INDEXES FROM transactions;
SHOW INDEX FROM transactions;
SHOW KEYS FROM transactions;

-- Drop an index
DROP INDEX idx_tx_date ON transactions;

-- Suggest, force, or ignore an index for a specific query
SELECT * FROM transactions USE INDEX (idx_tx_date) WHERE transaction_date > '2024-01-01';
SELECT * FROM transactions FORCE INDEX (idx_tx_date) WHERE transaction_date > '2024-01-01';
SELECT * FROM transactions IGNORE INDEX (idx_tx_date) WHERE transaction_date > '2024-01-01';
```

3.3 Real-World Uses and Drawbacks

Indexes are essential in production databases but come with trade-offs. The table below summarises the key considerations:

	Detail
Real-world uses	Improved customer experience (faster queries), operational efficiency (quick lookups), data analysis and reporting (aggregate queries on large tables run faster).
Drawback: Choosing indexes	Indexing a column with very few distinct values, such as <code>is_verified</code> which only holds 0 or 1, adds overhead without meaningful speed gains.
Drawback: Storage space	Each index stores a copy of the indexed data in a separate structure. More indexes means more disk space used.
Drawback: Maintenance overhead	Every <code>INSERT</code> , <code>UPDATE</code> , or <code>DELETE</code> must also update all related indexes, which slows down write operations on heavily indexed tables.

Assignment: Run `SHOW INDEXES FROM` any table after creating an index. Give a written description of each column that appears in the output.

4. Types of Database Tables

Tables in MySQL can be classified based on their longevity, storage mechanism, and purpose. Understanding the differences helps in choosing the right table type for a given task.

Table Type	Persists After Session?	Stored On Disk?	Best Used For
Base Table	Yes	Yes	All standard persistent data storage
Temporary Table	No	Yes (temp area)	Intermediate results within a single session
Memory (Heap) Table	No (lost on restart)	No (RAM only)	Caching and fast temporary lookups
Partitioned Table	Yes	Yes (split across files)	Very large tables that need performance optimisation
View	Yes (structure only)	No (no data stored)	Simplifying complex queries, restricting column access

4.1 Base Tables

Base tables are the standard, permanent tables created with `CREATE TABLE`. They persist across sessions and can be accessed and modified by any user with the appropriate permissions. All data remains until it is explicitly deleted or the table is dropped.

4.2 Temporary Tables

Temporary tables are created with the `TEMPORARY` keyword and only exist for the duration of the current session. They are automatically dropped when the connection is closed. They are very useful for storing intermediate results when performing complex multi-step calculations without affecting the original data.


```
-- Create a temporary table from a query result
CREATE TEMPORARY TABLE temp_active_users AS
SELECT * FROM users WHERE is_verified = 1;

-- Use it like a normal table
SELECT COUNT(*) FROM temp_active_users;

-- More advanced: temp table with HAVING
CREATE TEMPORARY TABLE IF NOT EXISTS dept_avg_salaries AS
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING COUNT(employee_id) > 5;
```

4.3 Memory (Heap) Tables

Memory tables reside entirely in RAM and are useful for very fast lookups. Because they are stored in memory and not on disk, all data is lost when the database server restarts. They are used for caching query results or storing session-specific data that changes frequently.

```
CREATE TABLE session_cache (
    id      INT AUTO_INCREMENT,
    value   VARCHAR(255),
    PRIMARY KEY (id)
) ENGINE=MEMORY;
```

4.4 Partitioned Tables

Partitioned tables split the data of a single large table across multiple storage files based on rules defined on a column. This improves performance and management for very large datasets. Queries that target a specific partition only scan that portion of the data rather than the entire table.

```
CREATE TABLE sales (
    sale_date  DATE          NOT NULL,
    product_id INT          NOT NULL,
    total_sales DECIMAL(19,2)
)
PARTITION BY RANGE (YEAR(sale_date)) (
    PARTITION p2021 VALUES LESS THAN (2022),
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025)
);
```

4.5 Views

A view is a virtual table defined by a stored SELECT query. It does not store any data itself. Every time the view is queried, the underlying SELECT is re-executed against the base tables. Views are useful for simplifying complex queries, hiding sensitive columns, and presenting a clean interface for reporting.

```
-- Create a view showing only successful transactions
CREATE VIEW v_successful_transactions AS
SELECT * FROM transactions WHERE status = 'SUCCESS';
```

```
-- Query the view like a normal table
SELECT * FROM v_successful_transactions ORDER BY amount DESC;

-- Drop a view
DROP VIEW IF EXISTS v_successful_transactions;
```

Key distinction: A *VIEW* always reflects current data because it re-runs its query each time. A *TEMPORARY TABLE* is a snapshot taken at creation time and does not update automatically.

5. Modifying Tables

After a table has been created and populated, there are two main SQL statements used to make changes: ALTER TABLE and UPDATE.

5.1 ALTER TABLE

ALTER TABLE is a DDL command used to change the structure of an existing table. It can add or remove columns, change data types, and add or remove constraints.

```
-- Add a new column
ALTER TABLE students ADD phone VARCHAR(15);

-- Change the data type of an existing column
ALTER TABLE students MODIFY COLUMN phone CHAR(15);

-- Rename a column
ALTER TABLE students RENAME COLUMN phone TO phone_number;

-- Drop a column
ALTER TABLE students DROP COLUMN phone_number;

-- Add a constraint after table creation
ALTER TABLE students ADD CONSTRAINT chk_gpa CHECK (gpa BETWEEN 0.00 AND 4.00);

-- Rename a table
ALTER TABLE students RENAME TO lmu_students;
```

5.2 UPDATE

UPDATE is a DML command used to modify the values stored in existing rows. It must almost always be used with a WHERE clause. Running UPDATE without WHERE will update every single row in the table.

```
-- Basic syntax
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;

-- Update one column for a specific row
UPDATE users SET balance = 750000.00 WHERE user_id = 1;

-- Update multiple columns at once
UPDATE users
SET balance = 500000.00, is_verified = 1
```

```
WHERE user_id = 5;

-- Best practice: always preview with SELECT before UPDATE
SELECT * FROM users WHERE user_id = 5;
-- Then run the UPDATE once you have confirmed the right rows
```

Warning: Always run a *SELECT* with the same *WHERE* clause before executing an *UPDATE* or *DELETE*. This confirms you are targeting the correct rows before making changes that cannot easily be reversed.

6. Data Modelling

Data modelling is the process of creating a visual representation of an information system to show how data will be stored and used in a business operation. The primary goal of data modelling is to ensure that data objects required by the organisation are represented accurately before any database is built.

Before modelling data, the requirements of the information system must be fully understood. The concept of abstraction guides how much detail is included at each stage. Abstraction is the process of reducing complexity by hiding unnecessary details and exposing only the essential features of a concept or system.

6.1 What is a Schema?

A schema is the structural design or blueprint of a database. It defines how data is organised and how the relationships among tables are associated. A schema outlines the tables, the fields in each table, the data types of those fields, and the relationships between tables. Think of it as the architectural plan of the database before any data is inserted.

In MySQL Workbench, a schema is equivalent to a database. When you create a new database with *CREATE DATABASE*, you are creating a new schema.

6.2 The Three Types of Data Models

Data modelling progresses through three levels of abstraction, each serving a different audience and purpose:

Model Type	Level of Abstraction	Audience	Components	Represented By
Conceptual	Highest	Business stakeholders and analysts	Entities, Attributes, Relationships	Basic ER Diagram
Logical	Medium	Database designers and developers	Tables, Columns, Primary and Foreign Keys, Normalisation	Detailed ER Diagram or UML
Physical	Lowest	Database administrators and developers	Physical storage, Indexing strategy, Performance optimisation, Exact SQL	SQL scripts, EER Diagram

- **Conceptual Data Model:** Focuses on the high-level structure of the data and fundamental business rules without considering technical implementations. It outlines entities and their relationships at the organisational level. This is always the first step in database design.

- **Logical Data Model:** Developed from the conceptual model by structuring data into database-specific schemas. It involves decisions on structure and constraints without yet considering the specific database system or hardware that will be used.
- **Physical Data Model:** The most detailed model. It translates the logical model into a technical implementation optimised for the selected DBMS (in our case, MySQL). It includes exact SQL scripts, index definitions, storage details, and partition strategies.

6.3 How the Models Are Achieved

Model	How It Is Created
Conceptual	Created through discussions and interviews with business stakeholders to capture high-level business needs and the scope of the system.
Logical	Developed by structuring the conceptual entities into tables, defining attributes, choosing primary and foreign keys, and applying normalisation rules.
Physical	Produced by translating the logical model into exact SQL CREATE TABLE statements, defining indexes, storage engines, and any performance optimisations specific to MySQL.

6.4 Schemas in Data Warehouse Design

In data warehousing and business intelligence, the term schema refers to the arrangement of fact and dimension tables. Two common patterns are the Star Schema and the Snowflake Schema.

	Star Schema	Snowflake Schema
Structure	One central fact table surrounded directly by dimension tables.	Dimension tables are further normalised into additional sub-tables.
Complexity	Simple. Fewer joins needed for queries.	More complex. More joins needed.
Query speed	Faster for reporting and dashboards.	Slower due to more joins, but more storage-efficient.
Best for	Reporting, dashboards, and OLAP queries.	Large data warehouses where storage efficiency matters.
Example	transactions at the centre, with users and agents as dimension tables directly connected.	agents table further split into agents and agent_regions sub-tables.

7. ER Diagrams

An Entity-Relationship (ER) diagram is a visual tool used to represent the conceptual data model of a system. It shows the entities in a system, their attributes, and the relationships between them.

Term	Definition
Entity	A real-world object or concept about which data is stored. Represented by a rectangle. Example: Student, Course, Transaction.
Relation	A table is a relation. It consists of tuples (rows) and attributes (columns).

Schema	The structural blueprint of the database defining tables, fields, data types, and relationships.
Attribute	A property that describes an entity. Represented by an oval in a conceptual ER diagram. Example: first_name, email, balance.
Relationship	The association between two or more entities. Represented by a diamond in a conceptual ER diagram.
Primary Key	An attribute that uniquely identifies each instance of an entity. Underlined in ER diagrams.
Foreign Key	An attribute in one entity that references the primary key of another entity. Enforces referential integrity.

7.1 Types of Relationships

- **One to One (1:1):** One record in an entity is associated with exactly one record in another entity, and vice versa. Example: a person owns one passport and a passport belongs to one person. In SQL, enforced by placing a FOREIGN KEY with a UNIQUE constraint on the dependent table.
- **One to Many (1:N):** One record in an entity can be associated with many records in another entity. This is the most common relationship. Example: one agent handles many transactions. In SQL, the foreign key is placed on the many-side table.
- **Many to Many (M:N):** Multiple records in one entity are associated with multiple records in another. Example: a user can subscribe to many savings plans and each plan can have many users. In SQL, this requires a junction table (also called a bridge or associative table) that holds the foreign keys of both entities.
- **Self-referencing:** A relationship within a single table where one row references another row in the same table. Example: in an Employees table, a manager is also an employee, so a manager_id column references the employee_id column in the same table.

7.2 Cardinality and Ordinality

- **Cardinality:** The numerical relationship between entities. It expresses the maximum number of times an entity can participate in a relationship. The values are 1:1, 1:N, or M:N.
- **Ordinality:** Whether participation in a relationship is mandatory or optional. It expresses the minimum number of times an entity must participate. In ER notation, a circle on the relationship line means optional (zero or more), while a bar means mandatory (one or more).

For example, in the relationship between a transaction and an agent: a transaction may optionally have an agent (ordinality = optional), but an agent can handle many transactions (cardinality = 1:N). The agent_id column in the transactions table is nullable, which reflects this optional participation.

7.3 Symbols in Conceptual ER Diagrams

Symbol	Shape	Represents
Entity	Rectangle	A real-world object or concept
Relationship	Diamond	The association between two entities
Attribute	Oval	A property of an entity
Primary Key attribute	Oval with underlined text	The identifying attribute of an entity
Cardinality	Number or symbol near line	How many instances participate

Ordinality (mandatory)	Bar on relationship line	Participation is required
Ordinality (optional)	Circle on relationship line	Participation is not required

7.4 Many to Many: The Junction Table

Because SQL does not allow a direct many-to-many relationship between two tables, a junction table must be created. The junction table holds the primary keys of both related tables as foreign keys, and its primary key is typically a composite key made up of both foreign key columns. This ensures no duplicate combinations can exist.

```
-- Many-to-many: users can subscribe to many savings plans
-- and each plan can have many users
CREATE TABLE user_subscriptions (
    user_id    INT    NOT NULL,
    plan_id    INT    NOT NULL,
    start_date DATE DEFAULT (CURRENT_DATE),
    amount     DECIMAL(12,2) DEFAULT 0.00,
    PRIMARY KEY (user_id, plan_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (plan_id) REFERENCES savings_plans(plan_id)
);
```

7.5 EER Diagrams in MySQL Workbench

MySQL Workbench supports Enhanced Entity-Relationship (EER) diagrams, which extend the basic ER diagram with additional features such as inheritance (subclasses and superclasses) and more detailed relationship notation. EER diagrams in Workbench can be created in two ways:

- Manually: Use File > New Model and add a diagram. Drag and drop tables onto the canvas and draw relationship lines between them.
- Reverse Engineer: Use Database > Reverse Engineer to automatically generate an EER diagram from an existing database. This reads the CREATE TABLE statements and FK definitions from the server and draws the diagram for you.

The Workbench EER diagram uses specific icons to show primary keys (a key symbol), regular columns (a diamond), and relationship lines with crow's foot notation to indicate cardinality and ordinality.

Summary

In this module, we covered the core building blocks of working with SQL tables and understanding how databases are designed. A table is defined by its columns, which must each have a data type and may have one or more constraints. The order when defining a column is always: name, then data type, then constraint. Constraints enforce data integrity at either column or table level.

We explored querying and filtering data using SELECT, DISTINCT, WHERE, BETWEEN, LIKE, IN, NOT IN, IS NULL, AND, OR, ORDER BY, LIMIT, and aggregate functions including COUNT, SUM, AVG, MAX, and MIN. Indexes speed up retrieval but add overhead to write operations and storage.

Tables can be base tables, temporary tables, memory tables, partitioned tables, or views. Each serves a different purpose and has different persistence behaviour. A view is a virtual table with no stored data, while a temporary table is a real table that disappears when the session ends.

Data modelling progresses from a conceptual model (entities and relationships only) through a logical model (tables, keys, and normalisation) to a physical model (exact SQL and performance optimisation). A schema is the structural blueprint of a database. In data warehousing, the Star Schema places a fact table at the centre with dimension tables around it, while the Snowflake Schema further normalises those dimensions into sub-tables.

ER diagrams represent the conceptual model visually. Entities are rectangles, relationships are diamonds, and attributes are ovals. Cardinality defines how many instances participate in a relationship. Ordinality defines whether participation is mandatory or optional. Many-to-many relationships are resolved through junction tables. EER diagrams in MySQL Workbench can be built manually or generated automatically through Reverse Engineer.